



Structure in C++

The standard C does not allow the struct data type to be treated like built-in types.

For example :-

~~We can't~~ add or subtract two complex no. in C

Another important limitation of C structures is that they do not permit data hiding. Means ~~the~~ structure members are **public members**.

One more limitation of C structure is that structure can have only variable as member.



Program:

```
struct book
{
    int bookid;
    char title[20];
    float price;
};
```

```
void main()
{
    book b1;
}
```

* In C++ writing **struct** keyword is optional

function in a Structure:

```
struct book
{
    int bookid;
    char title[20];
    float price;
};
```

Datatype ← book input()

```
{
    book b;
    cout << "Enter bookid, title and price of book";
    cin >> b.bookid >> b.title >> b.price;
    return(b);
}
```

```
void main()
{
    book b1;
    b1 = input(); → function call
}
```




As we know C++ is a superset of C.
So, C++ supports all the features of structures as defined in C.
But C++ has expanded its capabilities further to suits its
OOPs philosophy.

In C++, a structure can have both **variable** and **functions** as member.

By creating **structure** in C++ we also satisfy the OOPS principle of **encapsulation**.

Encapsulation refers to wrapping up of data (variables) and functions into a **single unit** (like class and structure).

Program: DOSBox



```
#include <iostream.h>
```

```
#include <conio.h>
```

```
struct book
```

```
{
```

```
    int bookid;
```

```
    char title[20];
```

```
    float price;
```

```
    void input()
```

```
{
```

```
        cout << "Enter bookid, title and price";
```

```
        cin >> bookid >> title >> price;
```

```
    }
```

```
    void display()
```

```
{
```

```
        cout << "\n" << bookid << " " << title << " " << price;
```

```
    }
```

```
};
```

```
void main()
```

```
{
```

```
    clrscr(); → It is used to clear the console screen
```

```
    book b1;
```

```
    b1.input();
```

```
    b1.display();
```

```
    getch();
```

```
}
```

→ As **display()** is an **member function** to call/access we have to use dot operator
It is used to capture a single character input from the user without displaying it on the screen.

or

It is often used to hold the console screen. In turbo C++ or similar environments, after running a program, the console window often closes immediately.